

CM3720 - MACHINE LEARNING

FORECASTING CALL WAITING TIMES FOR THE SRI LANKA GOVERNMENT INFORMATION CENTER



Identified Problem

- **The Sri Lanka Government Information Center (GLC) receives thousands of citizen inquiries daily for there contact number 1919/ 0114191919**
- **However:**
 - Call volumes fluctuate heavily
 - Operators are unevenly distributed across shifts
 - Knowledge Base (KB) coverage varies
 - Staffing hours change over time



Identified Problem

- **This results in:**
 - High waiting times
 - Operator overload
 - Inefficient resource planning



Identified Problem

Can we predict future waiting times based on operational factors ,and use those predictions to optimize staffing and shift scheduling?



Proposed Solution

Design a Machine Learning–based predictive system that analyzes historical call centre data and learns how staffing, KB usage, and call loads affect waiting times, then predict average waiting time (in seconds) per month.



Dataset Overview

Government Information Centre Monthly calling data:

2016–2021 (real data) +
2022–2025 (synthetic
extension)

118 Data Rows

https://data.gov.lk/search/field_tags/calls-34

The screenshot displays the OPENDATA Portal of Sri Lanka interface. On the left, there are four filter panels: 'Content Types' (Dataset (15)), 'Topics' (Information Technology & Cyber Security (14)), 'Tags' ((-) Calls), and 'Format' (csv (9), excel (9), pdf (9)). The 'Publisher' panel shows 'Information and Communication Technology Agency of Sri Lanka (Pvt)'. On the right, the search results section shows '1 to 10 of 15 results'. The search bar is empty, and the 'Sort by' dropdown is set to 'Date changed' and 'Order' is 'Descending'. Below the search bar, there are two results cards. The first card is titled 'Monthly Total Calls Received for GIC - 2020' and shows 'Monthly Total Calls Received for GIC - 2020' with a 'Tag (s):' of 'Calls' and a 'CSV' download button. The second card is titled 'Monthly Total Calls Received for GIC - 2021' and shows 'Monthly Total Calls Received for GIC - 2021' with a 'Category (s):' of 'Information Technology & Cyber Security' and a 'Tag (s):' of 'Calls', also with a 'CSV' download button.



Dataset Overview

DATA COLLECTION FORM (Jan - Dec 2020)												
	Jan	Feb	Mar	Apr	May	June	July	August	September	October	November	December
Calls												
Total no. of calls	178310	159377	141863			117936	144775	153592	158514	164435	153108	148584
No. of calls successfully answered	169672	151515	160084	133863	125430	125430	134836	122182	123717	108252	106588	107804
No. of calls answered within 30 seconds	134595	121747	106393			82891	101172	64107	60311	56183	53164	50220
Average waiting time in queue	00.00.20	00.00.19	00.00.25			00.00.15	00.00.20	00.00.38	00.00.42	00.00.52	00.00.43	00.00.44
No. of calls abandoned by the caller	8638	7862	16027			10159	9939	31410	34797	56204	46520	40780
Queries and Escalations												
Total no. of Queries	167808	149892	133351			105427	131555	118356	120526	97438	92550	92628
No. of queries addressed by the KB	135677	121205	160084	133863	125430	86555	106789	97475	98488	77557	72213	73163
Total no. of trouble tickets	948	383	578			167	250	314	355	293	112	72
No. of trouble tickets resolved	50	38	52			8	157	305	339	246	96	66
Coverage												
No. of Govt organizations hosting their info in the KB	327	327	327	327	327	327	327	327	327	327	327	327

https://data.gov.lk/search/field_tags/calls-34



Dataset Overview

A1 : ✕ ✓ <i>fx</i> Year																						
	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V
1	Year	Month	Total No o	No. of ope	No. of ope	No. of ope	No. of ope	No. of ope	No. of ope	No. of ope	No. of ope	No. of hou	Total no. o	No. of call	No. of call	Average w	No. of call	Total no. o	No. of que	Total no. o	No. of trou	No. of
2	2016	1	17	2	8	1	0	2	2	1	0	1227	182108	148708	94136	00:00:33	33400	145568	114410	9928	511	
3	2016	2	17	2	8	1	0	2	2	1	0	1227	158072	136620	84590	00:00:21	21452	134343	106522	7058	96	
4	2016	3	17	2	8	1	0	2	2	1	0	1227	164773	152380	123289	00:00:22	12393	149426	117416	6615	120	
5	2016	4	17	2	8	1	0	2	2	1	0	1227	133225	129037	121692	00:00:19	4188	125540	99051	3310	186	
6	2016	5	17	2	8	1	0	2	2	1	0	1227	210736	181385	123612	00:00:33	29351	173963	139760	10923	212	
7	2016	6	17	2	8	1	0	2	2	1	0	1227	141432	138452	134910	00:00:16	2980	136360	106468	4516	259	
8	2016	7	17	2	8	1	0	2	2	1	0	1227	134523	131539	127863	00:00:16	2984	130330	102588	3730	278	
9	2016	8	17	2	8	1	0	2	2	1	0	1227	148183	144982	142786	00:00:16	3201	143960	113109	3226	196	
10	2016	9	17	2	8	1	0	2	2	1	0	1227	158599	152981	141343	00:00:16	5618	151751	121530	3310	145	
11	2016	10	17	2	8	1	0	2	2	1	0	1227	152614	148443	142050	00:00:17	4171	146672	116790	3019	191	
12	2016	11	17	2	8	1	0	2	2	1	0	1227	154042	148270	130618	00:00:19	5772	145609	116861	2693	211	
13	2016	12	17	2	8	1	0	2	2	1	0	1227	158342	153053	136563	00:00:22	5289	150011	121485	2592	148	
14	2017	1	28	4	16	1	0	4	4	2	2	1664	158880	154936	148076	00:00:18	3944	145568	114410	9928	511	
15	2017	2	28	4	16	1	0	4	4	2	2	1664	192465	176173	119602	00:00:24	16292	173063	144032	2186	111	
16	2017	3	28	4	16	1	0	4	4	2	2	1664	151393	148124	143067	00:00:21	3269	145107	114403	2392	88	
17	2017	4	28	4	16	1	0	4	4	2	2	1664	117620	114355	109317	00:00:24	3265	110005	86049	1494	67	
18	2017	5	28	4	16	1	0	4	4	2	2	1664	152275	147203	137332	00:00:22	5072	141662	110755	1614	70	
19	2017	6	28	4	16	1	0	4	4	2	2	1664	153407	146715	130006	00:00:23	6692	140073	111619	2291	164	



Dataset Overview

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 118 entries, 0 to 117
Data columns (total 22 columns):
#   Column                                                                 Non-Null Count  Dtype
---  -
0   Year                                                                 118 non-null   int64
1   Month                                                                118 non-null   int64
2   Total No of operators                                                118 non-null   int64
3   No. of operators/shift: Shift 1 weekdays                            118 non-null   int64
4   No. of operators/shift: Shift 2 weekdays                            118 non-null   int64
5   No. of operators/shift: Shift 3 weekdays                            118 non-null   int64
6   No. of operators/shift: Shift 4 weekdays                            118 non-null   int64
7   No. of operators/shift: Shift 1 weekends                            118 non-null   int64
8   No. of operators/shift: Shift 2 weekends                            118 non-null   int64
9   No. of operators/shift: Shift 3 weekends                            118 non-null   int64
10  No. of operators/shift: Shift 4 weekends                            118 non-null   int64
11  No. of hours per week that the call center is fully staffed and operational 118 non-null   int64
12  Total no. of calls                                                    116 non-null   float64
13  No. of calls successfully answered                                    118 non-null   int64
14  No. of calls answered within 30 seconds                              116 non-null   float64
15  Average waiting time in queue                                         106 non-null   object
16  No. of calls abandoned by the caller                                  108 non-null   float64
17  Total no. of Queries                                                  116 non-null   float64
18  No. of queries addressed by the KB                                    118 non-null   int64
19  Total no. of trouble tickets                                          104 non-null   float64
20  No. of trouble tickets resolved                                       104 non-null   float64
21  No. of Govt organizations hosting their info in the KB               118 non-null   int64
```



Feature Engineering

Time Conversion

```
1 # conversion of time
2 # 00:01:50 -> 110 seconds
3 def time_to_seconds(x):
4     try:
5         parts = str(x).split(':')
6         return int(parts[0])*3600 + int(parts[1])*60 + int(parts[2])
7     except:
8         return np.nan
9
10 df['Avg_waiting_time_sec'] = df['Average waiting time in queue'].apply(time_to_seconds)
11 df['Avg_waiting_time_sec'] = df['Avg_waiting_time_sec'].fillna(df['Avg_waiting_time_sec'].median())
12 df = df.drop(columns=['Average waiting time in queue'])
```

Engineered Features

- $\text{KB_coverage_ratio} = \text{KB queries} / \text{Total queries}$
- $\text{Staffed_ratio} = \text{staffed hours} / 168 \text{ hours}$
- Total_ops_weekday, Total_ops_weekend
- $\text{Call_per_operator} = \text{total calls} / \text{total operators}$
- Outlier removal at 99th percentile



Feature Engineering

```
1 # Derived features
2
3 # KB coverage
4 df['KB_coverage_ratio'] = df['No. of queries addressed by the KB'] / df['Total no. of Queries']
5
```

```
1 # Fully staffed ratio
2 df['Staffed_ratio'] = df['No. of hours per week that the call center is fully staffed and operational'] / (24*7)
3
```

```
1 # Total operators per day
2 weekday_shifts = ['No. of operators/shift: Shift 1 weekdays',
3                  'No. of operators/shift: Shift 2 weekdays',
4                  'No. of operators/shift: Shift 3 weekdays',
5                  'No. of operators/shift: Shift 4 weekdays']
6
7 weekend_shifts = ['No. of operators/shift: Shift 1 weekends',
8                 'No. of operators/shift: Shift 2 weekends',
9                 'No. of operators/shift: Shift 3 weekends',
10                 'No. of operators/shift: Shift 4 weekends']
11
12 df['Total_ops_weekday'] = df[weekday_shifts].sum(axis=1)
13 df['Total_ops_weekend'] = df[weekend_shifts].sum(axis=1)
```



Feature Engineering

```
1 # Call load per operator]
2 df['Call_per_operator'] = df['Total no. of calls'] / (df['Total_ops_weekday'] + df['Total_ops_weekend'] + 1e-5)
```

```
1 # Drop original shift columns
2 df = df.drop(columns=weekday_shifts + weekend_shifts)
```

```
1 # Remove outliers in target
2 df = df[df['Avg_waiting_time_sec'] < df['Avg_waiting_time_sec'].quantile(0.99)]
```

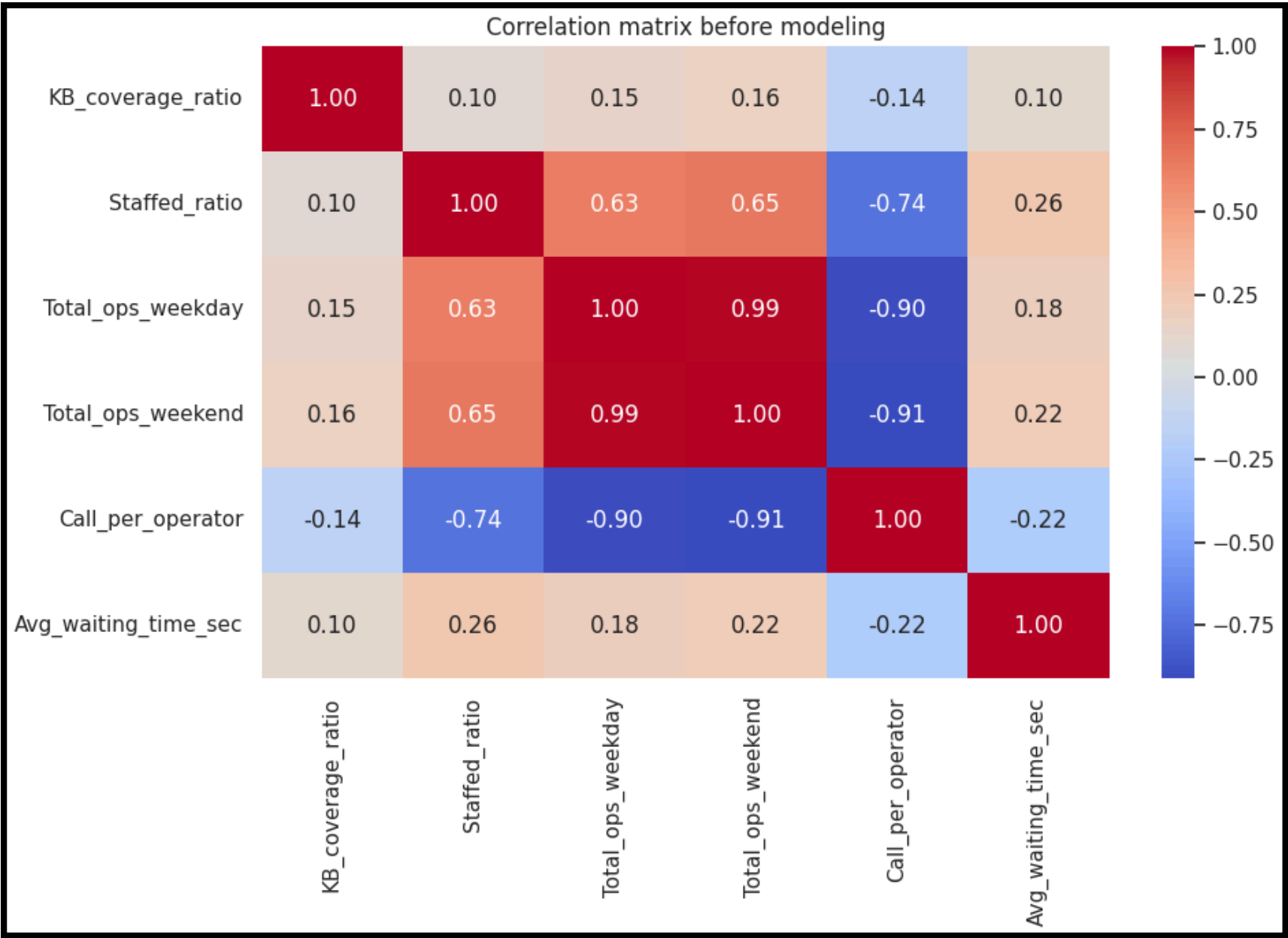
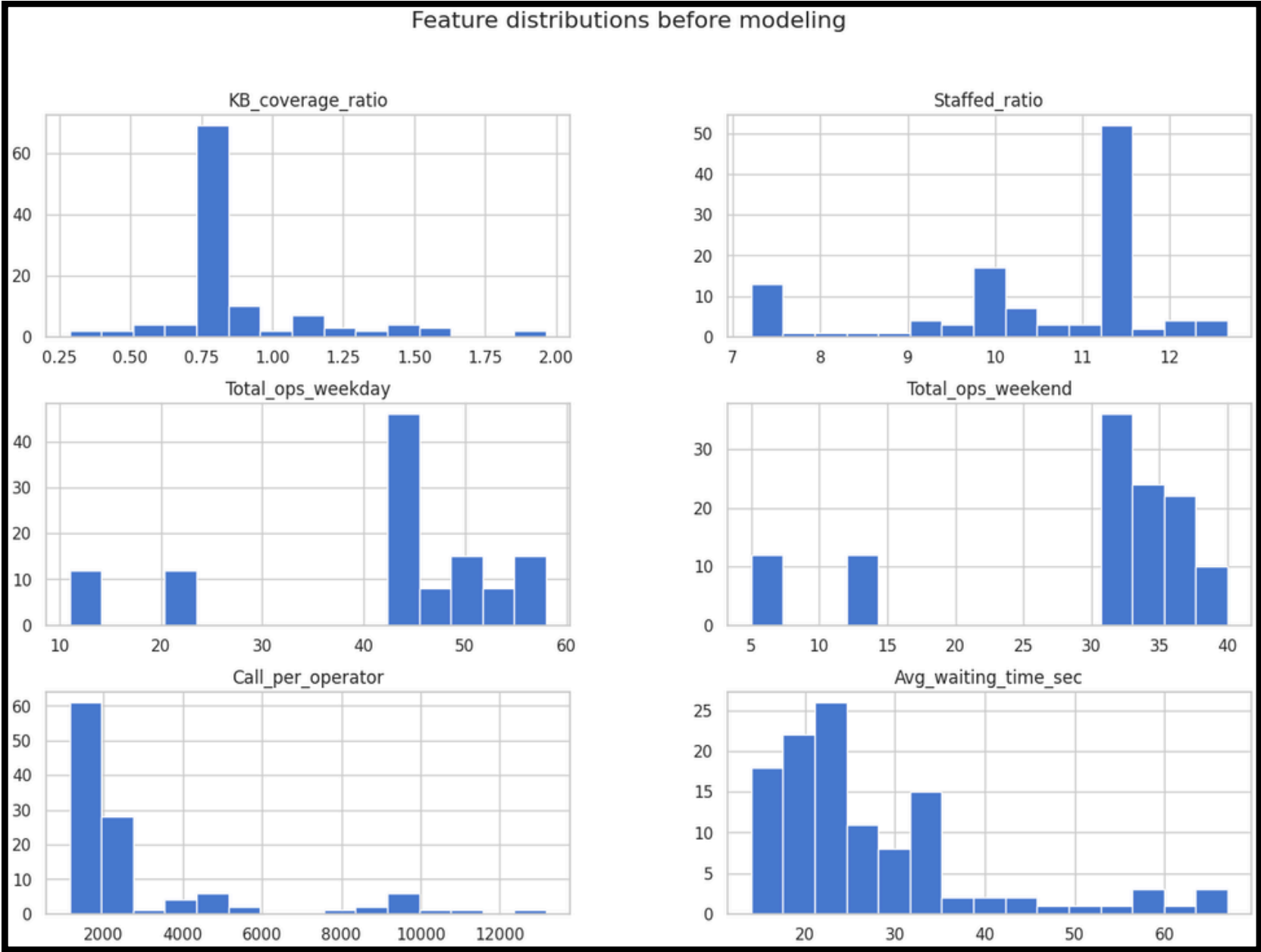


Features & Target

```
1 features = ['KB_coverage_ratio', 'Staffed_ratio',  
2            'Total_ops_weekday', 'Total_ops_weekend',  
3            'Call_per_operator']  
4  
5 target = 'Avg_waiting_time_sec'  
6
```



EDA



TRAINING

4 ensemble techniques were tested.



01

Bayesian Ridge Regression

- Chosen for stability & interpretability
- Good for small datasets



02

Gaussian Process Regression (GPR)

- Nonlinear kernel model



03

Support Vector Regression (SVR, RBF Kernel)

- Good for small-to-medium datasets
- Can model nonlinearity



04

LightGBM (Tree-based, Gradient Boosting)

- Powerful on tabular data



DEMONSTRATION

Optimizing Call Center Efficiency: Predictive Analysis of Sri Lanka GIC Call Records (2016–2025)

Project Description:

This project focuses on analyzing and predicting average waiting times for the Sri Lanka Government Information Center (GIC) based on call records from 2016–2021 (real data) and 2022–2025 (synthetic extension). By leveraging ensemble learning techniques, to understand how operational factors, such as staff allocation, shift distribution, and knowledge base coverage, impact caller waiting times.

Author : [Shiny Fernando] Date: [23 - 11 - 2025]

Import Libraries and Load Dataset

```
[224]  
✓ 0s  
1 import warnings  
2 warnings.filterwarnings('ignore')  
3  
4  
5 import numpy as np  
6 import pandas as pd  
7 import matplotlib.pyplot as plt  
8 import seaborn as sns
```



THANK YOU

